

Assignment 20.2

Shruthi Addagudi | 2303A52133 | Batch37

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Prompts Used:

Prompt1:

Generate a simple Python program for user registration and login that stores usernames and passwords. Keep it basic without using any encryption or hashing.

```
users = {}

def register():
    username = input("Enter username: ")
    if username in users:
        print("Username already exists!")
        return
    password = input("Enter password: ")
    users[username] = password
    print("Registration successful!")

def login():
    username = input("Enter username: ")
    if username not in users:
        print("Username not found!")
        return
    password = input("Enter password: ")
    if users[username] == password:
        print("Login successful!")
    else:
        print("Incorrect password!")
```

```
def main():
    while True:
        print("\n1. Register\n2. Login\n3. Exit")
        choice = input("Choose an option: ")
        if choice == "1":
            register()
        elif choice == "2":
            login()
        elif choice == "3":
            break
        else:
            print("Invalid choice!")

if __name__ == "__main__":
    main()
```

Prompt 2:

Analyze the following Python code and identify security issues related to password storage. Explain the risks of storing passwords in plain text.

```
import hashlib
import secrets

# DO NOT: Store passwords in plain text
# insecure_password = "mypassword123"

# SECURE: Hash passwords using a salt and strong algorithm
def hash_password(password: str) -> str:
    """Hash a password with a random salt using SHA-256."""
    salt = secrets.token_hex(16)
    pwd_hash = hashlib.pbkdf2_hmac('sha256', password.encode(), salt.encode(), 100000)
    return f"{salt}${pwd_hash.hex()}"

def verify_password(stored_hash: str, password: str) -> bool:
    """Verify a password against its stored hash."""
    salt, pwd_hash = stored_hash.split('$')
    new_hash = hashlib.pbkdf2_hmac('sha256', password.encode(), salt.encode(), 100000)
    return new_hash.hex() == pwd_hash

# Example usage
if __name__ == "__main__":
    user_password = "mypassword123"
    hashed = hash_password(user_password)
```

```
# Example usage
if __name__ == "__main__":
    user_password = "mypassword123"
    hashed = hash_password(user_password)
    print(f"Hashed password: {hashed}")
    print(f"Verification successful: {verify_password(hashed, user_password)}")
    print(f"Wrong password rejected: {verify_password(hashed, 'wrongpassword')}")
```

```
1. Register
2. Login
3. Exit
Choose an option: 2
Enter username: shein
Username not found!
```

Prompt 3:

Refactor the given Python code to store passwords securely using hashing (bcrypt or hashlib). Ensure proper user authentication is implemented.

```
import bcrypt
import hashlib
import os

class UserAuthentication:
    """Secure user authentication with hashed passwords"""

    def __init__(self):
        self.users = {}

    def hash_password_bcrypt(self, password: str) -> str:
        """Hash password using bcrypt"""
        salt = bcrypt.gensalt(rounds=12)
        hashed = bcrypt.hashpw(password.encode('utf-8'), salt)
        return hashed.decode('utf-8')

    def verify_password_bcrypt(self, password: str, hashed: str) -> bool:
        """Verify password against bcrypt hash"""
        return bcrypt.checkpw(password.encode('utf-8'), hashed.encode('utf-8'))

    def register_user(self, username: str, password: str) -> bool:
        """Register a new user with hashed password"""
        if username in self.users:
            print("User already exists!")
            return False

        self.users[username] = self.hash_password_bcrypt(password)
        print(f"User '{username}' registered successfully!")
        return True
```

```

class UserAuthentication:
    def authenticate_user(self, username: str, password: str) -> bool:
        """Authenticate user by verifying password"""
        if username not in self.users:
            print("User not found!")
            return False

        if self.verify_password_bcrypt(password, self.users[username]):
            print(f"User '{username}' authenticated successfully!")
            return True
        else:
            print("Invalid password!")
            return False

# Example usage
if __name__ == "__main__":
    auth = UserAuthentication()

    # Register users
    auth.register_user("alice", "secure_password123")
    auth.register_user("bob", "another_secure_pwd")

    # Authenticate
    auth.authenticate_user("alice", "secure_password123") # Success
    auth.authenticate_user("alice", "wrong_password") # Failure

```

```

1. Register
2. Login
3. Exit
Choose an option: 1
Enter username: shein
Enter password: 12233
Registration successful!

1. Register
2. Login
3. Exit
Choose an option: login shein
Invalid choice!

```

Explanation-

The AI-generated program initially stored passwords in plain text, which is a major security vulnerability. If the database is compromised, attackers can directly access user credentials. To improve security, password hashing techniques such as bcrypt or hashlib are used. Hashing converts passwords into irreversible encrypted forms, ensuring that even if data is leaked, the original passwords cannot be retrieved. This enhances authentication security and protects user data.

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and

Prompt 1 (Generate Insecure Code)

Generate a Python script that connects to an external API using a hardcoded API key.

```

1 import requests
2 import os
3
4 # Better practice: load API key from environment variable
5 api_key = os.getenv('API_KEY', 'your_api_key_here')
6
7 def fetch_data(endpoint):
8     """Fetch data from an external API"""
9     headers = {
10         'Authorization': f'Bearer {api_key}',
11         'Content-Type': 'application/json'
12     }
13
14     try:
15         response = requests.get(endpoint, headers=headers)
16         response.raise_for_status()
17         return response.json()
18     except requests.exceptions.RequestException as e:
19         print(f"API request failed: {e}")
20         return None
21
22 if __name__ == "__main__":
23     # Example: fetch from a public API
24     data = fetch_data('https://api.example.com/data')
25     if data:
26         print(data)

```

Prompt 2 (Security Analysis)

Identify the security risks of using hardcoded API keys in a Python program. Explain why this approach is unsafe.

```
def get_api_key_string():
```

```
    """
```

Identify security risks and unsafe practices of hardcoding API keys.

SECURITY RISKS OF HARDCODED API KEYS:

1. **Source Code Exposure**

- API keys become visible in version control history (Git, etc.)
- Keys persist in repositories even if later removed
- Anyone with repository access can extract credentials

2. **Unauthorized Access**

- Malicious actors can use exposed keys to impersonate your application
- Attackers gain direct access to backend services and resources
- Can lead to data theft, service abuse, and financial losses

3. **Difficult to Revoke**

- Keys embedded in code require redeployment to change
- Delayed response time during security incidents
- Multiple deployed instances may contain old compromised keys

4. **Accidental Leaks**

- Keys exposed through logs, error messages, and debugging output
- Shared in screenshots, documentation, or bug reports
- Visible in development environments and CI/CD pipelines

5. **Compliance Violations**

- Violates security standards (OWASP, PCI-DSS, SOC 2)
- Fails regulatory requirements for sensitive credential handling

6. **Supply Chain Risk**

- Dependencies or third-party code may extract hardcoded keys
- Build artifacts contain sensitive information

SAFE ALTERNATIVES:

- Use environment variables
- Deploy secrets management systems (AWS Secrets Manager, HashiCorp Vault)
- Implement IAM roles and temporary credentials
- Store keys in secure configuration files (gitignored)
- Use .env files for development (never commit to repository)

```
    """
```

```
    pass
```

Prompt 3 (Secure Refactoring)

Modify the Python script to securely access the API key using environment variables instead

```

1 import os
2 from dotenv import load_dotenv
3
4 # Load environment variables from .env file
5 load_dotenv()
6
7 # Securely retrieve the API key
8 api_key = os.getenv('API_KEY')
9
10 if not api_key:
11     raise ValueError("API_KEY environment variable is not set")
12
13 # Use the API key in your code
14 print(f"API Key loaded successfully")

```

Explanation-

The original script used hardcoded API keys, which is insecure because sensitive credentials can be exposed through source code sharing or version control systems. This can lead to unauthorized API usage and data breaches. The improved approach uses environment variables to store API keys securely. This ensures that sensitive information is not directly embedded in the code, improving confidentiality and making the application more secure and scalable.

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Prompt

Generate a simple Python program that performs encryption and decryption using a basic method like Base64 encoding.


```

import base64

def encrypt_message(message):
    """Encrypt a message using Base64 encoding."""
    encoded = base64.b64encode(message.encode('utf-8'))
    return encoded.decode('utf-8')

def decrypt_message(encrypted_message):
    """Decrypt a Base64 encoded message."""
    decoded = base64.b64decode(encrypted_message.encode('utf-8'))
    return decoded.decode('utf-8')

def main():
    print("=== Base64 Encryption/Decryption ===\n")

    # Get user input
    user_message = input("Enter a message to encrypt: ")

    # Encrypt
    encrypted = encrypt_message(user_message)
    print(f"Encrypted: {encrypted}\n")

    # Decrypt
    decrypted = decrypt_message(encrypted)
    print(f"Decrypted: {decrypted}")

if __name__ == "__main__":
    main()

```

Explanation-

The AI-generated program used weak encryption techniques such as Base64 encoding, which is not true encryption but simple encoding that can be easily reversed. This makes sensitive data vulnerable to attacks. The improved version uses strong encryption methods such as Fernet from the cryptography library, which provides secure symmetric encryption. This ensures that data can only be decrypted using the correct key, significantly enhancing data protection.

Task 4 – Real-Time Application: Security Review of AI-Generated

Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Prompt used:

Generate a simple Flask web application with a login system and database interaction.

Then perform a security review of the generated code:

1. Identify at least three vulnerabilities such as:

- SQL Injection
- Lack of input validation
- Hardcoded credentials

2. Improve the code by:

- Using parameterized queries
- Validating user inputs
- Removing hardcoded sensitive data

3. Provide:

- The improved secure version of the code
- Explanation of vulnerabilities found
- Explanation of the fixes applied

```

from flask import Flask, render_template, request, session
import sqlite3

app = Flask(__name__)
app.secret_key = "hardcoded_secret_key_12345"

DB_PATH = "users.db"

def init_db():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY,
            username TEXT,
            password TEXT
        )
    """)
    conn.commit()
    conn.close()

@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

```

```

# SQL Injection vulnerability
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"

conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
cursor.execute(query)
user = cursor.fetchone()
conn.close()

if user:
    session['user'] = username
    return "Login successful"
    return "Login failed"

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0')

```

Explanation-

The AI-generated web service contained multiple security vulnerabilities, including SQL injection, lack of input validation, and hardcoded credentials. SQL injection allows attackers to manipulate database queries, while missing

validation can lead to malicious input exploitation. Hardcoded credentials expose sensitive data. These issues were fixed by using parameterized queries, validating user inputs, and securely managing credentials. The improved version enhances application security, prevents unauthorized access, and ensures safe handling of user data.

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
operation = input("Enter operation (+, -, *, /): ")

if operation == '/':
    result = num1 / num2
elif operation == '+':
    result = num1 + num2
elif operation == '-':
    result = num1 - num2
elif operation == '*':
    result = num1 * num2

print(f"Result: {result}")
```

Explanation-

The initial program did not handle runtime errors, which could cause the application to crash when invalid input is provided or when division by zero occurs. This reduces reliability and user experience. By implementing try-except blocks, the program can catch and handle exceptions gracefully. It provides meaningful error messages and prevents abrupt termination, making the application more robust and user-friendly.